# Coin

## Architecture Decision Record

### ADR-001: Technical Stack & Infrastructure Alignment

coin.family

February 22, 2026

---

CONFIDENTIAL — INTERNAL TEAM ONLY

# Table of Contents

# 1. Context & Purpose

This Architecture Decision Record (ADR) aligns the team on key technical decisions for Coin — a peer-to-peer token exchange on Solana where humans and AI agents trade tokens directly with each other.

Two documents currently define Coin's technical direction:

- **PRD v2 + Product Roadmap** — The team-approved product specification defining the MVP (4-week agent-to-agent demo) and the post-funding V1 build (12 weeks to public launch). Focused on a P2P token exchange with escrow settlement on Solana.
- **Architecture Blueprint** (dEXploarer) — A comprehensive technical architecture document describing infrastructure, patterns, and technology choices. Scoped as a broader "fintech super-app" covering banking, cards, investments, payments, rewards, and social features.

These two documents have meaningful overlap *and* meaningful conflicts. This ADR resolves the conflicts, records the decisions, and ensures every team member is building on the same foundation.

| | |
|---|---|
| Status | Proposed — Pending Team Review |
| Authors | wes, dEXploarer |
| Reviewers | satsbased, 2AM, cody |
| Date | February 22, 2026 |
| Scope | MVP Phase 1 (Weeks 1–4) + V1 (Weeks 5–16) |
| References | PRD v2, Product Roadmap, Architecture Blueprint v1.0 |

> **How to read this document:** Each decision is presented as a card with Context (why the decision is needed), Options Considered (what we evaluated), Decision (what we chose), and Consequences (trade-offs and implications). Decisions marked Accepted are final for MVP. Decisions marked Deferred will be revisited post-funding.

# 2. Decision Summary

Quick reference for all decisions in this ADR. Details follow in subsequent sections.

| ID | Decision | Choice | Status |
|---|---|---|---|
| **001a** | Authentication Provider | Privy | Accepted |
| **001b** | Backend Architecture | Convex (data) + Hono (compute/webhooks) — no full microservices for MVP | Accepted |
| **001c** | Repository Structure | Multi-repo (5 repos) — evaluate monorepo at V2 | Accepted |
| **001d** | Cloud Platform & Hosting | Vercel (web) + Convex Cloud → Self-Hosted (backend) + Railway (agents + Hono) | Accepted |
| **001e** | QA & Production Environments | Devnet + Preview = QA; Mainnet + Production = Prod | Accepted |
| **001f** | Vector Database | None for MVP — revisit at V2+ | Deferred |
| **001g** | State Management & Patterns | Zustand (client) + Convex (server) + Zod validation | Accepted |

# 3. ADR-001a: Authentication Provider

### Authentication Provider

Accepted

CONTEXT

Coin needs authentication that supports social login (X / Farcaster / email), embedded Solana wallets (users never see seed phrases), and MPC or enclave-based key management. The PRD specifies Privy. The Architecture Blueprint specifies Clerk.

OPTIONS CONSIDERED

**Privy (Selected)**

- Native Solana embedded wallets
- MPC + enclave-based key management
- X (Twitter) + Farcaster + email login
- Social recovery (no seed phrases)
- Built for crypto-native apps
- Wallet auto-created on signup
- Free tier supports MVP

**Clerk (Blueprint)**

- SOC 2 Type 2 certified
- Bot protection + breach detection
- 10K MAU free tier
- No native Solana wallet support
- No embedded crypto wallets
- Would require separate wallet provider
- Built for traditional web apps

DECISION

**Use Privy for authentication and embedded wallets.** Coin is a crypto-native product where every user needs a Solana wallet. Privy provides auth + wallet as a single integration. Using Clerk would require bolting on a separate wallet provider (Turnkey, Crossmint, or raw keypair management), adding complexity with no benefit.

CONSEQUENCES

- Single SDK handles both auth and wallet provisioning
- Users sign in with one tap — no MetaMask, no seed phrases
- Privy handles key recovery via social login
- Trade-off: Privy lacks Clerk's bot protection and breach detection features (mitigated by Cloudflare WAF at edge if needed later)

**Open Decision (Phase 2a):** The PRD notes that the final wallet provider choice (Privy embedded vs Phantom embedded vs Turnkey vs Crossmint) will be confirmed during a Phase 2a technical spike. Privy is the default unless the spike reveals a blocker.

# 4. ADR-001b: Backend Architecture

**Backend Architecture** — Accepted — Revised Feb 23

CONTEXT

Both the PRD and the Architecture Blueprint agree on Convex as the primary backend. The blueprint additionally proposes 6 Hono-based microservices deployed on Fly.io. Team feedback (dEXploarer) argues that relying solely on Convex Cloud is too limiting — a Hono server layer alongside Convex provides more control over compute, webhooks, and long-lived connections.

DECISION

**Convex as the primary data layer + a thin Hono server for compute that doesn't fit serverless.** Not 6 independent microservices — one Hono process alongside Convex, handling the workloads listed below.

## Responsibility Split: Convex vs Hono

Convex (Data Layer — Primary)

- Offer CRUD (create, accept, cancel)
- Real-time subscriptions (marketplace feed)
- User data, agent config, reputation
- Scheduling (cron jobs, cleanup)
- File storage (KYC docs, avatars)
- ACID transactions for state changes
- Optimistic updates for UI

Hono (Compute Layer — Companion)

- Webhook receivers (Privy, Solana tx confirmations)
- Agent health checks & admin API
- WebSocket fan-out for price feeds
- Long-running compute (batch operations)
- External API orchestration
- Rate limiting & request validation
- Any future PCI-isolated processing

## Blueprint Microservices — MVP Mapping

| Blueprint Microservice | Coin Equivalent | MVP Decision |
|---|---|---|
| Trading Engine (Hono) | On-chain escrow + Convex offer management | Convex + escrow program (no separate service) |
| Payment Processor | N/A — all settlement is on-chain via USDC | Not applicable |
| Compliance Engine | US geo-blocking via Vercel Edge Middleware | Middleware only — no dedicated service |
| Notification Service | Convex real-time subscriptions | Built into Convex |
| Market Data Service | Jupiter WebSocket price feeds | Hono layer in coin-agents handles this |
| Analytics Service | Not in MVP scope | Deferred to V2 |

**Why this split works:** Convex handles all data operations, real-time sync, and ACID guarantees. Hono handles everything that needs raw HTTP control, long-lived connections, or sits outside Convex's serverless model. This avoids the overhead of 6 independent microservices while still giving us a server layer for compute. The Hono process runs on Railway alongside the agent workers — no additional infrastructure.

WHEN TO SPLIT INTO SEPARATE SERVICES

The single Hono process can be split into dedicated microservices in V2+ if: (a) a feature like fiat on-ramp requires PCI-isolated processing, (b) a Python-based ML workload (fraud detection) needs its own runtime, or (c) independent scaling is needed for a specific compute workload. Until then, Convex + one Hono server covers everything.

# 5. ADR-001c: Repository Structure

**Repository Structure** Accepted

CONTEXT

The Architecture Blueprint proposes a single Turborepo + pnpm monorepo. The team has already created 5 separate repositories under the `coin-agent-exchange` GitHub organization.

DECISION

**Keep the multi-repo structure for MVP and V1. Evaluate monorepo migration for V2.**

## Current Repository Layout

| Repository | Technology | Purpose |
|---|---|---|
| `coin-web` | Next.js + Tailwind | Consumer web application |
| `coin-convex` | Convex (TypeScript) | Backend: schema, queries, mutations, actions |
| `coin-escrow` | Anchor / Rust | Solana escrow program (on-chain settlement) |
| `coin-agents` | TypeScript / Node.js | Agent runtime workers (market makers) |
| `coin-prd` | HTML / PDF | Product docs (PRD, roadmap, ADRs) |

### Multi-Repo (Selected for MVP)

- Already set up with GitHub org, labels, issues, and project board
- Independent deploy cycles per repo
- Simple CI/CD — no Turborepo config needed
- Clear ownership boundaries per component
- Works well for a 5-person team

### Monorepo (Blueprint, evaluate at V2)

- Shared types across all packages
- Atomic cross-repo changes
- Turborepo remote caching
- Better for larger teams (10+)
- Higher initial setup cost

TYPE SHARING STRATEGY (MULTI-REPO)

To share TypeScript types between `coin-web`, `coin-convex`, and `coin-agents` without a monorepo, publish a `@coin/types` npm package (private, on GitHub Packages) or use Git submodules for a shared `types/` directory. Decision on the exact mechanism deferred to Sprint 1.

# 6. ADR-001d: Cloud Platform & Hosting

**Cloud Platform & Hosting** — Accepted — Revised Feb 23

CONTEXT

The PRD/roadmap specifies free-tier infrastructure for MVP (<$50/mo) scaling to ~$4K/mo post-funding. The blueprint proposes Vercel + Fly.io + Convex Cloud + Cloudflare. Team feedback (dEXploarer) raised an important point: **self-hosting Convex on Railway** provides a layer of data privacy, which matters for a crypto financial product handling wallet data and trade history. Convex is now fully open-source and has a one-click Railway deploy template.

DECISION

**MVP: Convex Cloud (zero ops, ship fast). V1: Migrate to self-hosted Convex on Railway (data sovereignty).** Vercel for web, Railway for agents + Hono + self-hosted Convex, Helius for RPC. Add Cloudflare when needed for DDoS/WAF.

## Hosting Map

| Component | MVP (Free Tier) | V1 Post-Funding | Blueprint Equivalent |
|---|---|---|---|
| **Web App** (coin-web) | Vercel Hobby ($0) | Vercel Pro ($20/mo) | Vercel — aligned |
| **Backend** (coin-convex) | Convex Cloud Starter ($0) | Self-hosted Convex on Railway + Postgres (~$20–40/mo) | Convex — aligned (self-hosted for privacy) |
| **Agents + Hono** (coin-agents) | Railway Hobby ($5/mo) | Railway Pro ($20/mo) | Blueprint uses Fly.io |
| **Escrow** (coin-escrow) | Solana Devnet (free) | Solana Mainnet | N/A in blueprint |
| **RPC** | Helius Free (100K/day) | Helius Growth | Not specified |
| **CDN / WAF** | None (Vercel handles CDN) | Cloudflare (if needed) | Cloudflare |
| **Monitoring** | Vercel Analytics (free) | Sentry + Vercel Analytics | Sentry + Grafana + Prometheus |
| **CI/CD** | GitHub Actions (free) | GitHub Actions | GitHub Actions — aligned |

## Monthly Cost Projection

| Phase | Monthly Cost | Notes |
|---|---|---|
| MVP (Weeks 1–4) | **<$50/mo** | All free tiers + $50–200 one-time mainnet seed |
| V1 (Weeks 5–16) | **~$4,000/mo** | Vercel Pro, self-hosted Convex + Postgres on Railway, Helius Growth, Railway (agents + Hono) |
| V2+ (Blueprint level) | **$8,000–15,000/mo** | Fly.io microservices, Cloudflare Pro, ClickHouse, Redis, monitoring stack |

> **Why Railway over Fly.io for agents:** The agent workers are long-running Node.js processes (subscribing to WebSocket feeds, posting offers). Railway's developer experience is simpler for this use case — deploy from GitHub, automatic restarts, built-in logging. Fly.io's Machines API is more powerful but adds operational complexity that isn't justified for 2–20 agent processes.

## Self-Hosted Convex Migration Plan

> **Amendment (Feb 23) — based on team feedback from dEXploarer:** Self-hosting Convex provides data sovereignty over user wallet data, trade history, and financial activity. For a crypto product, controlling your own data infrastructure is a meaningful privacy and security advantage — especially for enterprise and institutional users down the road.

Convex is fully open-source (FSL Apache 2.0) and supports self-hosting via Docker with SQLite or Postgres as the storage backend. Railway provides a one-click deploy template.

| | Convex Cloud (MVP) | Self-Hosted on Railway (V1) |
|---|---|---|
| **Setup** | Zero config | Docker + Postgres (one-click Railway template) |
| **Scaling** | Automatic | Manual (single machine; scale vertically on Railway) |
| **High Availability** | Managed by Convex team | Team-managed (Railway auto-restarts + Postgres replication) |
| **Data Privacy** | Convex Cloud servers (US) | Your infrastructure — full data sovereignty |
| **Cost** | $0 (free tier) → $25/mo (Pro) | ~$20–40/mo (Railway compute + Postgres) |
| **Backups** | Managed | Team-configured (Railway S3 backup template available) |
| **Migrations** | Automatic | Manual SQL migrations between versions |
| **Dashboard** | Hosted at dashboard.convex.dev | Self-hosted (open-source dashboard included) |

### Migration Path

1. **MVP (Weeks 1–4):** Use Convex Cloud free tier. Zero ops, ship the demo fast.
2. **V1 Phase 1 (Week 5):** Deploy self-hosted Convex on Railway using Docker + Postgres. Migrate schema and data from Cloud.
3. **V1 Phase 2+:** Run agents, Hono server, and self-hosted Convex all on Railway. Single platform, full control.

4. **Migration is low-risk:** Same Convex API, same client libraries. Only the backend URL changes in `.env`.

# 7. ADR-001e: QA & Production Environments

### QA & Production Environments

Accepted

CONTEXT

Neither the PRD nor the blueprint explicitly defines separate QA and production environments. For a pre-funding team of 5, we need a lightweight environment strategy that prevents shipping broken code without the overhead of a full staging infrastructure.

DECISION

**Two-environment strategy leveraging Solana's built-in devnet/mainnet split and Vercel's preview deployments.**

## Environment Map

| Layer | Development / QA | Production |
|---|---|---|
| **Solana Network** | Devnet (free SOL, no real value) | Mainnet (real USDC, real tokens) |
| **Web App** | Vercel Preview (auto per PR) | Vercel Production ( `main` branch) |
| **Convex Backend** | Convex Dev project | Convex Production project |
| **Agents** | Local / Railway Dev environment | Railway Production |
| **Escrow Program** | Deployed on Devnet | Deployed on Mainnet |
| **RPC** | Helius Devnet endpoint | Helius Mainnet endpoint |

## Deployment Flow

1. Developer pushes feature branch → Vercel auto-generates Preview URL
2. PR reviewed by at least one team member + all CI checks pass (lint, typecheck, tests)
3. Merge to `main` → Vercel auto-deploys to Production + Railway deploys agents
4. Escrow program upgrades: tested on Devnet first, deployed to Mainnet only after full team sign-off

**V1 Upgrade (Post-Funding):** Add a formal staging environment with a dedicated Convex staging project, Railway staging service, and Solana Devnet. The blueprint's canary deployment pattern (5% traffic → 25% → 50% → 100%) is good practice but overkill until we have real user traffic to split. Adopt it in V2.

# 8. ADR-001f: Vector Database

### Vector Database (Pinecone vs Qdrant)

Deferred

CONTEXT

The Architecture Blueprint includes Pinecone for "AI-powered search, recommendation engine embeddings." This supports features like robo-advisory, smart transaction categorization, and fraud detection. The question was raised: should Coin use Pinecone or Qdrant?

DECISION

**Neither. No vector database is needed for MVP or V1. Deferred to V2+.**

Coin's MVP and V1 do not include any features that require vector embeddings:

- No AI-powered search — token pairs are a fixed list of 14
- No recommendation engine — marketplace shows all offers
- No robo-advisory — users and agents trade directly
- No fraud detection ML — escrow program + spending limits handle safety
- No smart categorization — all trades are token-for-USDC

## If/When We Need One

| Option | Type | Pros | Cons |
|---|---|---|---|
| **Convex Vector Search** | Built-in | Zero additional infrastructure, already in our stack, type-safe | Less mature than dedicated vector DBs, limited tuning options |
| **Pinecone** | Managed SaaS | Zero ops, fast, widely adopted, free tier available | Vendor lock-in, cost scales with vector count |
| **Qdrant** | Self-hosted or Cloud | Open source, more control, cheaper at scale | Requires ops/hosting, smaller ecosystem |

**Recommendation for V2+:** Try Convex's built-in vector search first. It's already in the stack and eliminates an additional service. If the use case outgrows it (millions of embeddings, advanced filtering), migrate to Pinecone for managed simplicity.

# 9. ADR-001g: State Management & Patterns

**State Management & Development Patterns** Accepted

CONTEXT

The Architecture Blueprint recommends several development patterns that are independent of the product scope. These are good engineering practices that apply regardless of whether Coin is a P2P exchange or a fintech super-app.

DECISION

**Adopt the following patterns from the Architecture Blueprint:**

| Pattern | Blueprint Recommendation | Coin Adoption |
|---|---|---|
| **Client State** | Zustand + Convex reactive queries | Adopt — Zustand for UI state, Convex for server state |
| **Validation** | Zod schemas on every boundary | Adopt — shared Zod schemas for forms, API, and Convex |
| **Forms** | React Hook Form + Zod | Adopt — for offer creation, trade preferences, settings |
| **Styling** | Tailwind CSS 4 + shadcn/ui | Adopt — consistent with PRD wireframes |
| **Convex Organization** | queries.ts / mutations.ts / actions.ts per domain | Adopt — clean separation for coin-convex |
| **Feature Modules** | Vertical slices (components, hooks, actions, stores) | Adopt — for marketplace, wallet, agent features in coin-web |
| **Component Hierarchy** | Primitives → Composites → Features → Pages | Adopt — design system structure for coin-web |
| **Error Boundaries** | App → Layout → Feature → Component | Adopt — graceful degradation at each level |
| **Optimistic Updates** | Convex built-in optimistic update support | Adopt — instant feedback for offer creation/acceptance |

**Not adopted from the blueprint:** LaunchDarkly feature flags (overkill for 5-person team — use environment variables), ClickHouse analytics DB (no analytics workload yet), Redis/Upstash cache (Convex handles caching), Pulumi IaC (3 services don't need infrastructure-as-code), Infisical secrets (use environment variables + Vercel/Railway secret management).

# 10. Blueprint vs PRD/Roadmap — Full Alignment Matrix

This table compares every major dimension of the Architecture Blueprint against what the PRD and Roadmap define for Coin. Where they conflict, the decision column records which direction we're going.

| Dimension | PRD / Roadmap | Architecture Blueprint | Decision |
|---|---|---|---|
| **Product Scope** | P2P token exchange on Solana | Fintech super-app (banking, cards, investments, payments, rewards, social) | PRD scope for MVP/V1 |
| **Auth** | Privy | Clerk | Privy (ADR-001a) |
| **Backend** | Convex | Convex + 6 Hono microservices | Convex (data) + Hono (compute) (ADR-001b revised) |
| **Repo Structure** | 5 separate repos | Turborepo monorepo | Multi-repo (ADR-001c) |
| **Web Framework** | Next.js + Tailwind | Next.js 15 + Tailwind 4 + shadcn/ui | Aligned — adopt shadcn/ui |
| **Mobile** | Not in MVP; V2 React Native | React Native + Expo from Phase 0 | Defer to V2 per PRD |
| **Hosting** | Vercel + Railway | Vercel + Fly.io + Convex Cloud | Vercel + Railway + self-hosted Convex (ADR-001d revised) |
| **State Management** | Not specified | Zustand + Convex | Adopt (ADR-001g) |
| **Blockchain** | Solana (Anchor escrow, Jupiter feeds) | Multi-chain (Ethereum, Bitcoin, Solana, Polygon, etc.) | Solana only for MVP/V1 |
| **Trade Settlement** | On-chain escrow (Anchor/Rust) | Not defined (generic "crypto trading") | PRD escrow model |

| Dimension | PRD / Roadmap | Architecture Blueprint | Decision |
|---|---|---|---|
| **AI Agents** | Core concept — market-making agents | Not mentioned | PRD agent model |
| **Token List** | 14 Solana tokens vs USDC | Generic multi-chain crypto | PRD token list |
| **Vector DB** | Not mentioned | Pinecone | Deferred (ADR-001f) |
| **Validation** | Implied | Zod everywhere | Adopt Zod (ADR-001g) |
| **KYC/AML** | Geo-blocking only for V1 | Full compliance engine (Jumio, Onfido, Chainalysis) | Geo-blocking for V1, full KYC in V3+ |
| **Build Timeline** | 4-week MVP + 12-week V1 | 24-week full build (Phases 0–5) | PRD timeline |

**Key Takeaway:** The Architecture Blueprint is a valuable *north-star vision* document. It describes where Coin *could* go. But for MVP and V1, we build what the PRD defines — a focused P2P exchange — using the engineering patterns and practices from the blueprint. Scope follows the PRD; engineering quality follows the blueprint.

# 11. Patterns Adopted from the Blueprint

While the blueprint's product scope goes beyond MVP, several of its engineering recommendations are strong and should be adopted immediately. These patterns improve code quality and set up a clean foundation that scales.

## Convex Function Organization

Organize `coin-convex` by domain, with separate files for queries, mutations, and actions:

```
convex/
  functions/
    offers/queries.ts     # getOffers, getOffersByToken, getMyOffers
    offers/mutations.ts   # createOffer, cancelOffer, acceptOffer
    offers/actions.ts    # settleOnChain, syncEscrowStatus
    agents/queries.ts    # getAgents, getAgentPerformance
    agents/mutations.ts  # registerAgent, updateConfig
    prices/queries.ts    # getLatestPrices, getPriceHistory
    prices/mutations.ts  # updatePrice
  schema.ts
  crons.ts
  http.ts
```

## Feature Module Pattern for coin-web

Each feature in the web app follows the vertical slice pattern:

```
features/
  marketplace/
    components/    # OfferCard, OfferTable, BuySellToggle
    hooks/        # useOffers, useMarketPrice
    stores/       # marketplaceStore (Zustand)
    validators/   # offerSchema (Zod)
  wallet/
    components/    # WalletBalance, TokenList, DepositFlow
    hooks/        # useWallet, useBalance
  agent/
    components/    # AgentStatus, AgentConfig, AgentPnL
    hooks/        # useAgent, useAgentMetrics
```

## Design System Hierarchy

Build the `coin-web` component library in layers, using shadcn/ui as the primitive foundation:

**L1** **Primitives** (shadcn/ui) — Button, Input, Select, Dialog, Badge, Skeleton, Card

**L2** **Composites** — PriceDisplay, TokenSelector, OfferRow, SpreadBadge, WalletCard

**L3** **Features** — MarketplaceFeed, OfferCreator, AgentDashboard, TradeHistory

**L4** **Pages** — BuyersMarket, SellersMarket, WalletView, AgentConfig, Settings

# 12. Next Steps

This ADR is proposed and pending team review. The following actions are needed to finalize it.

## Immediate (This Week)

1. **Team Review:** All 5 partners review this document and flag disagreements in Discord.
2. **Decision Lock:** Any unresolved conflicts are discussed in a team call. Once consensus is reached, the status of this ADR moves from `Proposed` to `Accepted`.
3. **Begin MVP Sprint 1:** With architecture aligned, development starts per the roadmap's Week 1 plan (Anchor escrow program + Convex schema + Jupiter price integration).

## Before V1 Kickoff (Post-Funding)

1. **ADR-002:** Wallet provider spike — confirm Privy embedded wallets vs alternatives.
2. **ADR-003:** Type sharing strategy — npm package vs Git submodules vs monorepo migration.
3. **ADR-004:** Formal staging environment setup + CI/CD pipeline definition.

## Decision Review Cadence

Architecture decisions are living documents. Each ADR will be reviewed at phase boundaries (end of MVP, end of V1 Phase 1, etc.) and updated if the technical landscape has changed. The Architecture Blueprint remains a reference for long-term direction — decisions deferred today may be revisited as the product grows.

---

> **Questions?** Drop them in `#dev-architecture` on Discord. Tag @wes or @dEXploarer for technical clarifications.

---